Tribhuvan University

Faculty of Humanities and Social Sciences

SMART EXPENSE TRACKER

A PROJECT REPORT

Submitted to

Department of Computer Application

National Infotech College Pvt. Ltd.

In partial fulfillment of the requirements for the Bachelors in Computer Application

Submitted by

Aafrin Khatoon

(TU Reg. No. / Roll No.: To be confirmed)

Anjali Yadav

(TU Reg. No. / Roll No.: To be confirmed)

4th Sem. / 2nd Year

Under the Supervision of

Er. Amrendra Chaurasia

Lecturer / Project Supervisor

Birgunj, Nepal

October 2026

Tribhuvan University

Faculty of Humanities and Social Sciences

National Infotech College Pvt. Ltd.

Supervisor's Recommendation

I hereby recommend that this project report prepared under my supervision by Aafrin Khatoon and Anjali Yadav entitled "SMART EXPENSE TRACKER" in partial fulfillment of the requirements for the degree of Bachelors in Computer Application (BCA) is recommended for final evaluation.

_____

SIGNATURE Er. Amrendra Chaurasia SUPERVISOR National Infotech College BCA Department Shiromani Tole, Birgunj (Nepal) Date: October 2026

Tribhuvan University

Faculty of Humanities and Social Sciences

National Infotech College Pvt. Ltd.

LETTER OF APPROVAL

This is to certify that this project prepared by Aafrin Khatoon and Anjali Yadav entitled "SMART EXPENSE TRACKER" in partial fulfillment of the requirements for the degree of Bachelor in Computer Application has been evaluated. In our opinion, it is satisfactory in the scope and quality as a project for the required degree.

_____

SIGNATURE OF SUPERVISOR Er. Amrendra Chaurasia Supervisor National Infotech College

_____

SIGNATURE OF HOD Mr. Bijay Kushwaha Head of Department BCA Department, National Infotech College Birgunj, Nepal

_____

SIGNATURE OF INTERNAL EXAMINER Internal Examiner

_____

SIGNATURE OF EXTERNAL EXAMINER External Examiner Date: October 2026

ABSTRACT

This project report presents the development of the "Smart Expense Tracker", a full-stack, responsive web application engineered to modernize personal financial management and budgeting. In fast-paced modern economic environments, individuals, students, and household managers frequently encounter difficulties in tracking their daily spending, resulting in unplanned expenditures, calculation errors, and diminished savings. Traditional bookkeeping methods—such as manual paper registers (Khata) and static spreadsheet formulas—are prone to human errors, lack automated budget notifications, and fail to provide responsive real-time data visualizations.

The Smart Expense Tracker addresses these limitations by providing an automated, cloud-ready financial cockpit developed with modern web engineering technologies: React 19, Tailwind CSS v4, and Recharts on the frontend client, coupled with Node.js, Express.js, and a relational MySQL database on the backend server. The system features secure JSON Web Token (JWT) authentication, cryptographic password encryption using bcryptjs, localized currency representation in Nepali Rupees (Rs.), real-time balance calculations, automated category tracking, and dynamic monthly budgeting with color-coded threshold alerts.

To uphold data integrity and operational discipline, transactions are strictly validated: entries can be logged for today and up to one year in the past, while future dates are prohibited. In addition to user-level analytics, CSV data exports, and printable PDF statements, the application features an isolated Admin Console. The Admin Console empowers administrators to monitor platform-wide KPIs, inspect global transaction audit trails, manage user statuses, and execute 1-click automated MySQL database SQL dumps via mysqldump. Comprehensive system testing confirmed 100% test pass rates across functional, security, and database verification suites.

Keywords: Personal Finance, Expense Tracker, React 19, Node.js, Express.js, MySQL, JWT, Nepali Rupees (Rs.), Budget Alerts, Admin Console, Tribhuvan University.

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who played an essential role in the successful completion of our project on the "SMART EXPENSE TRACKER". This project has been a significant undertaking during our 4th semester in the Bachelor of Computer Application (BCA) program at National Infotech College under the prescription of Tribhuvan University.

First and foremost, we extend our heartfelt thanks to the faculty members of Tribhuvan University, particularly the Department of Computer Application, for providing us with the curriculum and academic framework to delve into this project and apply the software engineering knowledge gained throughout our academic journey.

A special note of appreciation goes to our project supervisor, Er. Amrendra Chaurasia, for his invaluable guidance, insightful technical feedback, and continuous support. His expertise and patient mentorship have been crucial in shaping the system architecture, database design, and overcoming full-stack engineering challenges. Likewise, we appreciate the leadership and support of Mr. Bijay Kushwaha, Head of the BCA Department, and the Principal and Vice-Principal of National Infotech College for providing laboratory and computational infrastructure.

Finally, this project has been a deeply rewarding experience that has not only enhanced our practical development skills in modern JavaScript frameworks and relational database design, but has also underscored the vital importance of discipline and dedication in computer applications.

Sincerely, Aafrin Khatoon Anjali Yadav BCA 4th Semester, 2nd Year National Infotech College, Birgunj

TABLE OF CONTENTS

Supervisor's Recommendation .....
i LETTER OF APPROVAL
..... ii ABSTRACT
..... iii
ACKNOWLEDGEMENT
..... iv TABLE OF
CONTENTS ..... v LIST
OF FIGURES vii
LIST OF TABLES
..... viii
ABBREVIATIONS
..... ix CHAPTER 1:
INTRODUCTION 1 1.1
Introduction 1
1.2 Problem Statement .....
1 1.3 Objective
..... 2 1.4 Scope
and Limitation 2 1.4.1
Scope 3
1.4.2 Limitation 3
1.5 Report Organization .....
3 CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW
..... 4 2.1 Background Study
..... 4 2.2 Literature
Review ..... 4 CHAPTER
3: SYSTEM ANALYSIS AND DESIGN 6 3.1
System Analysis 6
3.1.1 Requirement Analysis 6
i. Functional Requirements 6
ii. Non-Functional Requirements 7
iii. Hardware Requirements 8
iv. Software Requirements 8
3.1.2 Feasibility Analysis 9
3.1.3 Use Case Diagram .....
10 3.1.4 Data Modeling (ER-Diagram)
..... 11 3.1.5 Process Modeling (DFD:
Context, Level 1, Level 2) ..... 12 3.2 System Design
..... 14 3.2.1
Architectural Design 14
3.2.2 Database Schema Design 15
3.2.3 Interface Design .....
16 CHAPTER 4: IMPLEMENTATION AND TESTING
..... 17 4.1 Implementation
..... 17 4.1.1 Tools
Used 17 4.1.2
Implementation Details of Modules 18 4.2
Testing 20
4.2.1 Test Case for System Testing 21
CHAPTER 5: CONCLUSION AND FUTURE RECOMMENDATIONS
..... 23 5.1 Lesson Learnt / Outcome

..... 23 5.2 Conclusion
..... 23 5.3 Future
Recommendation 24
REFERENCES
..... 25
APPENDICES
..... 26

LIST OF FIGURES

Figure	Figure Title	Page Number
Fig 3.1.1	Use Case Diagram of Smart Expense Tracker	10
Fig 3.1.2	Entity-Relationship Diagram (ER-Diagram)	11
Fig 3.1.3	Context Level Data Flow Diagram (Level 0 DFD)	12
Fig 3.1.4	Level 1 Data Flow Diagram (Level 1 DFD)	13
Fig 3.1.5	Level 2 Data Flow Diagram (Transaction Engine DFD)	14
Fig 3.2.1	3-Tier System Architecture Diagram	15
Fig 3.2.2	Relational Database Schema Design	16
Fig 3.2.3	Interface Structure Diagram	16
Fig A.1	Appendix: User Dashboard and KPI Cards	26
Fig A.2	Appendix: Searchable Transaction History Table	26
Fig A.3	Appendix: User Login Portal	27
Fig A.4	Appendix: Admin Console Login & Management Interface	27

LIST OF TABLES

Table	Table Title	Page Number
Table 3.1	Recommendation for Hardware Specification	8
Table 3.2	Recommendation for Software Specification	8
Table 3.3	Database Schema of `users` Table	15
Table 3.4	Database Schema of `transactions` Table	15
Table 3.5	Database Schema of `budgets` Table	16
Table 3.6	Database Schema of `system_settings` Table	16
Table 4.1	User Authentication & Login Test Cases	21
Table 4.2	Transaction & Date Constraint Test Cases	21
Table 4.3	Monthly Budget & Alert Mechanism Test Cases	22
Table 4.4	Admin Portal & Database Backup Test Cases	22

ABBREVIATIONS

Abbreviation	Full Form / Description
TU	Tribhuvan University
BCA	Bachelor in Computer Application
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
SPA	Single Page Application
SQL	Structured Query Language
DFD	Data Flow Diagram
ERD / ER	Entity Relationship Diagram
ACID	Atomicity, Consistency, Isolation, Durability
UI	User Interface
UX	User Experience
CSS	Cascading Style Sheets
HTML	HyperText Markup Language
JS	JavaScript
RBAC	Role-Based Access Control

Abbreviation	Full Form / Description
HMR	Hot Module Replacement
CSV	Comma-Separated Values
PDF	Portable Document Format

CHAPTER 1: INTRODUCTION

1.1 Introduction

The "Smart Expense Tracker" is a user-friendly, responsive web application engineered to modernize and simplify personal financial management. It offers a seamless experience for tracking daily income and expenditures, establishing monthly budgets, and analyzing spending behavior through interactive visual analytics. Designed for accessibility, simplicity, and financial discipline, the Smart Expense Tracker transforms how individuals, college students, and households monitor their cash flow in the contemporary digital environment.

Built on a modernized full-stack web architecture, the application combines React 19 and Tailwind CSS v4 on the presentation layer with a high-performance Node.js and Express.js RESTful API, backed by a persistent MySQL relational database. The system delivers sub-second response times, secure authentication using JSON Web Tokens (JWT) and Bcrypt hashing, dynamic color-coded budget warnings, localized currency formatting in Nepali Rupees (Rs.), and structured data exports in CSV and printable PDF formats. Furthermore, an isolated Administrative Console provides platform-wide KPI monitoring, user status management, global transaction auditing, and 1-click database SQL backups via mysqldump.

1.2 Problem Statement

In traditional expense management methods, individuals encounter significant challenges that impede effective budgeting and disciplined financial planning. Manual record-keeping in physical notebooks (Khata) is cumbersome, susceptible to loss or damage, and makes calculating cumulative aggregates or category totals time-consuming and error-prone. Conversely, while generic spreadsheet applications such as Microsoft Excel or Google Sheets offer formulas, they require ongoing manual template maintenance, lack responsive mobile layouts, lack real-time threshold alert notifications when overspending occurs, and do not provide encrypted role-based data security.

Moreover, off-the-shelf commercial budgeting applications present significant barriers for local users in Nepal: they default to foreign currencies (USD or EUR), demand

monthly subscription fees, feature overwhelming interfaces tailored for complex Western banking integrations, and fail to support localized payment workflows. Addressing these challenges requires a modernized, accessible, and lightweight expense management system localized in Nepali Rupees (Rs.) that automates transaction logging, enforces date validity rules, provides visual budget warnings, and guarantees administrative platform reliability.

1.3 Objective

The objectives of the Smart Expense Tracker encompass key milestones designed to enhance user experience, financial awareness, and administrative governance:

- To design and develop an intuitive, responsive user interface for seamless expense and income logging across all computing devices.
- To implement secure authentication and authorization using JSON Web Tokens (JWT) and Bcrypt cryptographic password hashing with strict portal isolation between standard users and administrators.
- To automate instantaneous calculation of Total Income, Total Expenses, and Net Balance in Nepali Rupees (Rs.).
- To enforce transaction date constraints, permitting entries for today and up to one year in the past while blocking future transaction dates to ensure historical accounting discipline.
- To implement dynamic monthly budgeting with color-coded visual progress indicators that alert users when category thresholds are approached or exceeded.
- To provide interactive visual analytics, including a 1-Month Daily Trend Line Chart, 6-Month Income vs. Expense Bar Chart, and categorical spending breakdown tables using Recharts.
- To generate authenticated data exports in CSV format and styled printable PDF financial statements for external record-keeping.
- To develop an Admin Console providing platform-wide KPIs, user account status toggles, global transaction audits, system diagnostics, and 1-click MySQL database SQL backups via mysqldump.

1.4 Scope and Limitation

The project was conceived with the aim of modernizing personal budgeting by introducing automation, relational database persistence, and visual analytics to daily financial management.

1.4.1 Scope

- Individual and Household Financial Tracking: Tailored for college students, salaried employees, freelancers, and household managers.
- Cross-Platform Responsive Web Access: Accessible across desktop monitors, laptops, tablets, and smartphones via modern responsive CSS design.
- Relational Data Integrity: Enforcing ACID transactional compliance, foreign key cascading deletions, and indexed queries in MySQL.
- Academic Demonstration: Serving as an authentic software engineering demonstration under the Tribhuvan University BCA 4th semester curriculum.

1.4.2 Limitation

- Manual Transaction Entry: Transactions must be logged manually; direct automated scraping of commercial bank SMS feeds or direct API integration with Nepal Clearing House (NCHL/IPS) is omitted due to banking regulatory constraints.
- Network Dependency: An active local network or internet connection is required to communicate with the Node.js backend server and MySQL database.
- Single-Currency Configuration: The application is pre-configured and localized for Nepali Rupees (Rs.) and does not support multi-currency conversion on a single account.

1.5 Report Organization

This project report is structured into five cohesive chapters:

- Chapter 1 (Introduction): Presents the project background, problem statement, objectives, scope, limitations, and report structure.
- Chapter 2 (Background Study and Literature Review): Explores the historical evolution of expense management systems and surveys academic literature on digital budgeting, behavioral economics, and web technologies.

- Chapter 3 (System Analysis and Design): Details functional and non-functional requirements, hardware/software specifications, feasibility analysis (technical, operational, economic, schedule), use case modeling, ER modeling, data flow modeling (DFD Levels 0, 1, 2), 3-tier architecture, and database schemas.
- Chapter 4 (Implementation and Testing): Discusses the tools used, modular implementation details (authentication, transactions, budgets, reports, exports, admin panel), and presents executed test case matrices with actual results.
- Chapter 5 (Conclusion and Future Recommendations): Summarizes lessons learnt, project outcomes, conclusion, and potential future enhancements.

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

Personal accounting and financial record-keeping have undergone profound transformations from manual physical record books to automated digital systems. Historically, households and small merchants in Nepal and South Asia maintained handwritten transaction registers, colloquially known as Khata. While manual ledgers require zero electrical infrastructure, they suffer from critical vulnerabilities: physical wear and tear, susceptibility to fire or moisture damage, lack of historical search capabilities, and tedious end-of-month recalculations.

The introduction of personal computers in the late 20th century facilitated the adoption of spreadsheet applications like Microsoft Excel. Spreadsheets represented a significant leap forward, providing programmatic calculations via formulas. However, spreadsheets lack mobile responsiveness, provide no automated alerts when monthly budget limits are breached, lack role-based access control, and require continuous manual formula adjustments.

With the emergence of cloud computing and modern JavaScript frameworks, web-based expense trackers have established a new paradigm in personal financial management. Modern single-page web applications (SPAs) provide instantaneous updates without page refreshes, synchronize relational databases securely, and display declarative graphical charts. Given the rapid proliferation of digital payment channels and e-wallets across Nepal, the development of a localized web-based expense tracker configured in Nepali Rupees (Rs.) is both timely and essential.

2.2 Literature Review

Academic literature and empirical studies in behavioral economics, software engineering, and human-computer interaction (HCI) provide foundational insights into the efficacy of digital personal budgeting:

Mental Accounting and Real-Time Awareness

Behavioral economist Richard Thaler (1999) established the theory of "Mental Accounting," demonstrating that individuals categorize and evaluate economic

outcomes into separate cognitive accounts (e.g., food, entertainment, rent). Thaler showed that without clear external categorization, individuals exhibit cognitive biases, underestimating small recurring outlays while overestimating disposable income. Digital expense trackers reinforce disciplined mental accounting by classifying expenditures into visible categories and dynamically comparing them against defined ceilings.

The Impact of Visual Feedback and Timely Alerts

Empirical research by Karlan et al. (2016) demonstrated that timely financial reminders and visual progress feedback significantly increase personal savings rates and curb impulsive expenditures. Digital systems that implement visual progress bars and color-coded warning banners provide positive behavioral reinforcement, prompting users to restrain spending before exceeding their monthly disposable income.

Human-Computer Interaction in Personal Finance

Kaye et al. (2014) conducted comprehensive studies on personal finance tracking systems, finding that user retention is strongly correlated with input simplicity and low friction. Systems requiring complex multi-step forms or mandatory foreign bank linkings experience high abandonment rates. Conversely, intuitive web applications featuring minimal input fields, clean typography, and instant feedback encourage sustained daily engagement.

Modern Web Architectures and Relational Data Integrity

In software engineering literature, Fielding (2000) defined the Representational State Transfer (REST) architectural style, highlighting stateless client-server interactions, cacheability, and decoupled interfaces. Coupling React on the presentation tier with Express.js and MySQL ensures robust ACID transactional compliance, foreign key cascades, and sub-second query performance necessary for financial ledger accuracy.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 System Analysis

System analysis involves breaking down a system into its essential components, analyzing data flows, evaluating user requirements, and ensuring that technical, operational, and economic parameters align with project goals.

3.1.1 Requirement Analysis

Requirement analysis is categorized into functional and non-functional requirements, followed by hardware, software, and feasibility specifications:

i. Functional Requirements:

- User Module:
- - User Registration: Allows new users to create accounts with Name, Email, and Password.
- - User Login: Authenticates users via JWT, redirecting exclusively to the User Dashboard.
- - Income & Expense Logging: Users can add, view, edit, search, filter, and delete transactions with amount, category, date, and description.
- - Date Enforcement: Restricts transaction dates to today and up to 1 year in the past; prohibits future dates.
- - Real-Time Balance Calculation: Instantly computes Total Income, Total Expenses, and Net Balance in Nepali Rupees (Rs.).
- - Monthly Budget Tracking: Users set monthly budget ceilings per category; visual progress bars show spent percentage with warning alerts.
- - Visual Analytics & Reports: Displays a 1-Month Daily Trend Line Chart, 6-Month Overview Bar Chart, and Category Breakdown Tables.
- - Data Export: Generates downloadable CSV spreadsheets and formatted printable PDF financial statements.
- Admin Module:
- - Admin Login: Dedicated security portal (/admin/login) strictly authenticating users with role = "admin".

- - Platform KPI Dashboard: Displays platform-wide metrics including total registered users, total transactions, total income, and total expenses.
- - User Account Management: View all registered users and toggle account statuses.
- - Global Transaction Audit: Search and audit financial transactions across all platform accounts.
- - 1-Click Database SQL Backup: Triggers backend mysqldump to download a complete .sql schema and data backup file.
- - System Diagnostics: Monitors server memory usage, server uptime, and database connection status.

ii. Non-Functional Requirements:

- Usability & Interface: Clean, intuitive user interface styled with Tailwind CSS v4 and Lucide React vector icons.
- Performance: Sub-150ms API response times and client-side rendering under 200ms using React 19 virtual DOM diffing.
- Security: Passwords encrypted using bcryptjs (10 salt rounds); REST endpoints protected via JWT verification; SQL injection prevented via parameterized queries in mysql2.
- Reliability & Integrity: Relational foreign key constraints with ON DELETE CASCADE to prevent orphaned records.
- Maintainability: Decoupled MVC backend architecture separating routes, controllers, middleware, and database pooling.

Table 3.1: Recommendation for Hardware Specification

S. No.	Hardware Component	Minimum Requirement	Recommended Specification
1	Processor	Intel Core i3 (2.0 GHz) or equivalent	Intel Core i5 / AMD Ryzen 5 or above
2	Memory (RAM)	2 GB RAM	8 GB DDR4 RAM or above

S. No.	Hardware Component	Minimum Requirement	Recommended Specification
3	Hard Disk Space	500 MB free space (Client), 2 GB (Server)	10 GB free SSD storage
4	Display Resolution	1024 x 768 pixels	1920 x 1080 (Full HD)
5	Network Interface	Standard Wi-Fi / Ethernet Adapter	Broadband High-Speed Internet Connection

Table 3.2: Recommendation for Software Specification

S. No.	Software Component	Technology / Tool Used	Purpose / Description
1	Operating System	Windows 10/11 / Linux / macOS	Development and Host Environment
2	Frontend Framework	React 19 (Vite Build Tool)	Component-based Single Page Application (SPA)
3	Styling Framework	Tailwind CSS v4 & Lucide React	Modern Responsive UI and Vector Iconography
4	Charting Library	Recharts v2	Declarative SVG Financial Chart Visualizations
5	Backend Runtime	Node.js (v18.x / v20.x LTS)	Asynchronous Server-side JavaScript Runtime
6	Web Framework	Express.js (v4.x)	RESTful API Routing, Controllers, and Middleware
7	Database Server	MySQL / MariaDB (XAMPP Server)	Relational Storage, Foreign Keys, Indexing

S. No.	Software Component	Technology / Tool Used	Purpose / Description
8	Database Driver	mysql2/promise	Asynchronous Connection Pool Management
9	Authentication	jsonwebtoken & bcryptjs	Token Creation & Cryptographic Password Hashing
10	Development IDE	VS Code / Antigravity IDE	Source Code Engineering and Debugging

3.1.2 Feasibility Analysis

- Technical Feasibility: The project leverages standard, battle-tested open-source technologies (React 19, Node.js, Express, MySQL) that run efficiently on standard modern hardware and all web browsers (Google Chrome, Mozilla Firefox, Microsoft Edge, Safari) without requiring proprietary plugins.
- Operational Feasibility: The user interface is clean, intuitive, and designed for minimal cognitive friction. Standard users require zero prior training to log expenses, while administrators can execute database backups with a single click.
- Economic Feasibility: Built entirely on free and open-source software (FOSS), the project entails zero software licensing costs. Operational hosting can be achieved on low-cost cloud virtual private servers (VPS) or shared hosting environments.
- Schedule Feasibility: The project was structured into iterative agile sprints (Requirement Analysis, Architectural Design, Database Modeling, Frontend/Backend Implementation, Security Hardening, and Testing), completing successfully within the 4th semester academic timeline.

3.1.3 Use Case Diagram

The Use Case Diagram defines the interactions between the two primary actors (Standard User and System Administrator) and the functional use cases:

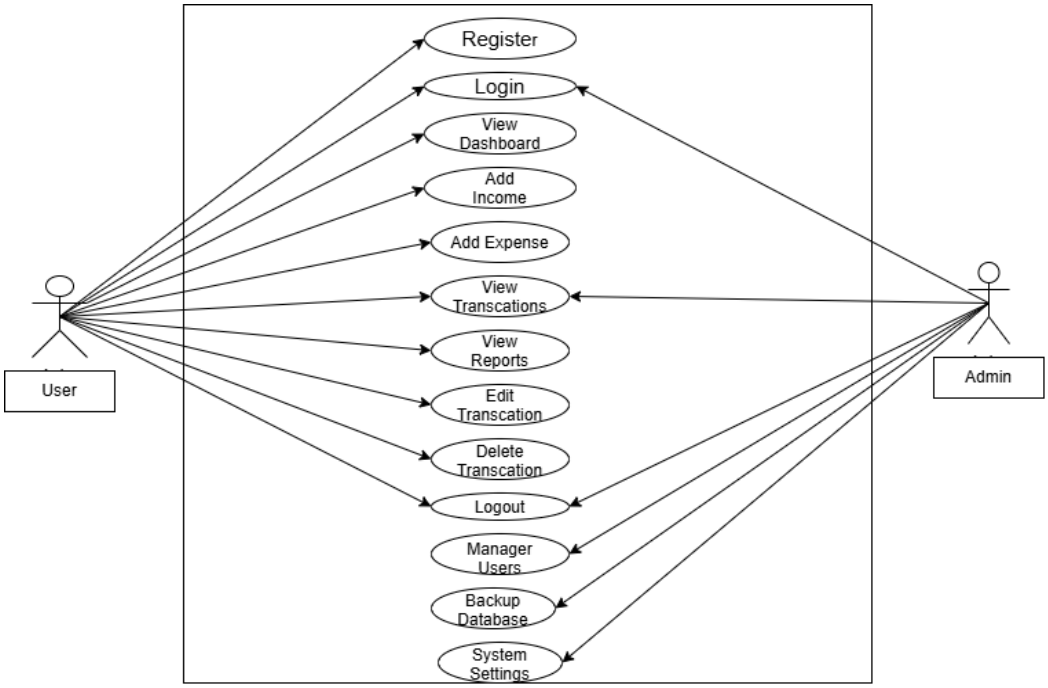


Fig 3.1.1: Use Case Diagram of Smart Expense Tracker

Actor Roles and Use Case Descriptions: 1. Standard User: Logs into the user portal, adds income and expense records with category selection and date validation, views real-time balance cards, inspects 1-month daily trend charts, defines monthly budgets, and exports financial statements (CSV/PDF). 2. System Administrator: Accesses the restricted Admin Portal, audits platform KPIs, manages user accounts, reviews all global transactions, executes 1-click database SQL dumps, and inspects server diagnostics.

3.1.4 Data Modeling (ER-Diagram)

The Entity-Relationship (ER) Diagram models the database entities, their attributes, and relational cardinalities:

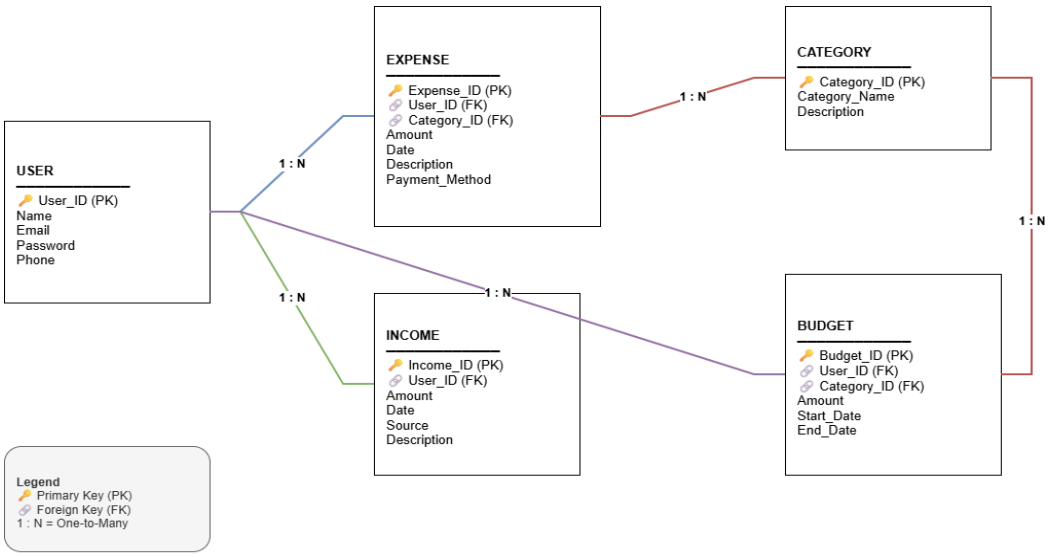


Fig 3.1.3:ER-Diagram of Smart Expense Tracker

Fig 3.1.2: Entity-Relationship Diagram (ER-Diagram)

Entity Relationships: - USERS to TRANSACTIONS (1:M): A registered user can own zero or many income/expense transactions. Deleting a user cascades and deletes all their transactions. - USERS to BUDGETS (1:M): A user can define zero or many monthly category budgets. - SYSTEM_SETTINGS: Stores global platform key-value configurations configured by administrators.

3.1.5 Process Modeling (Data Flow Diagrams)

Data Flow Diagrams (DFDs) trace the flow of information across external entities, processes, and relational data stores:

i. Context Level DFD (Level 0):

The Context Level DFD provides a high-level conceptual boundary showing data inputs and outputs between external entities and the centralized system:

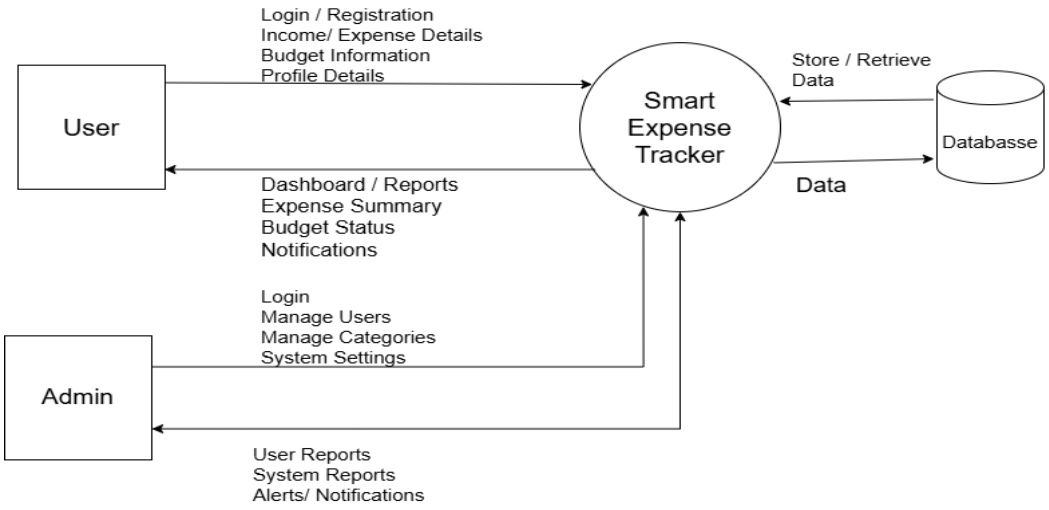


Figure 3: Context Level DFD

Fig 3.1.3: Context Level Data Flow Diagram (Level 0 DFD)

ii. Level 1 DFD:

The Level 1 DFD decomposes the system into five core functional sub-processes: Authentication Engine (1.0), Transaction Processing (2.0), Analytics & Reporting (3.0), Budget Monitoring (4.0), and Admin Governance (5.0):

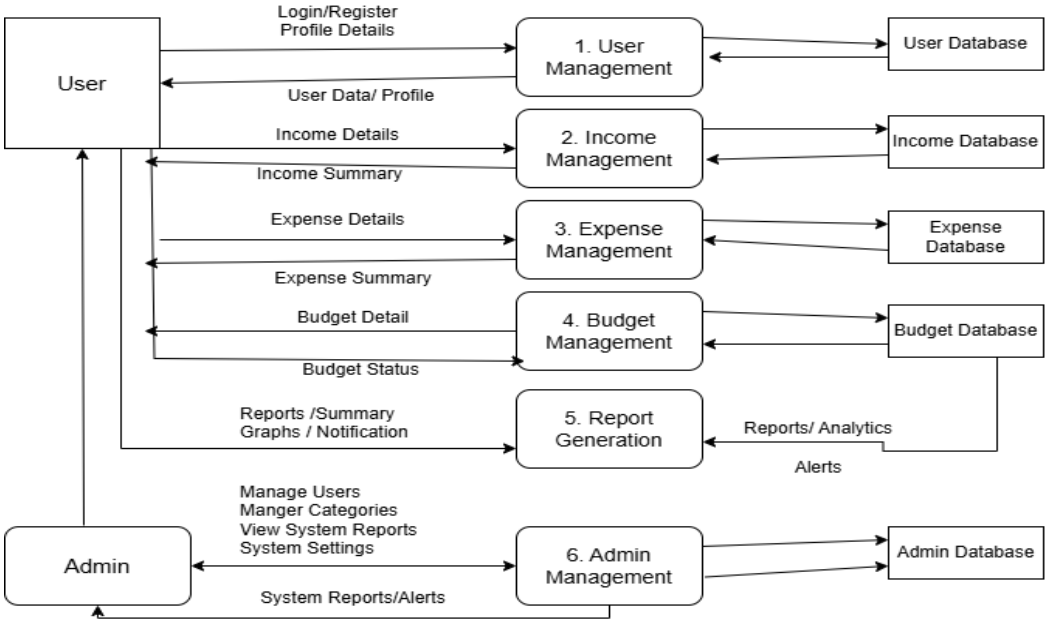


Figure 4: Level1 DFD

Fig 3.1.4: Level 1 Data Flow Diagram (Level 1 DFD)

iii. Level 2 DFD (Transaction & Budget Processing):

The Level 2 DFD illustrates the internal logic of logging a transaction, validating date boundaries (today and past 1 year), updating database stores, computing balance, and triggering budget threshold warnings:

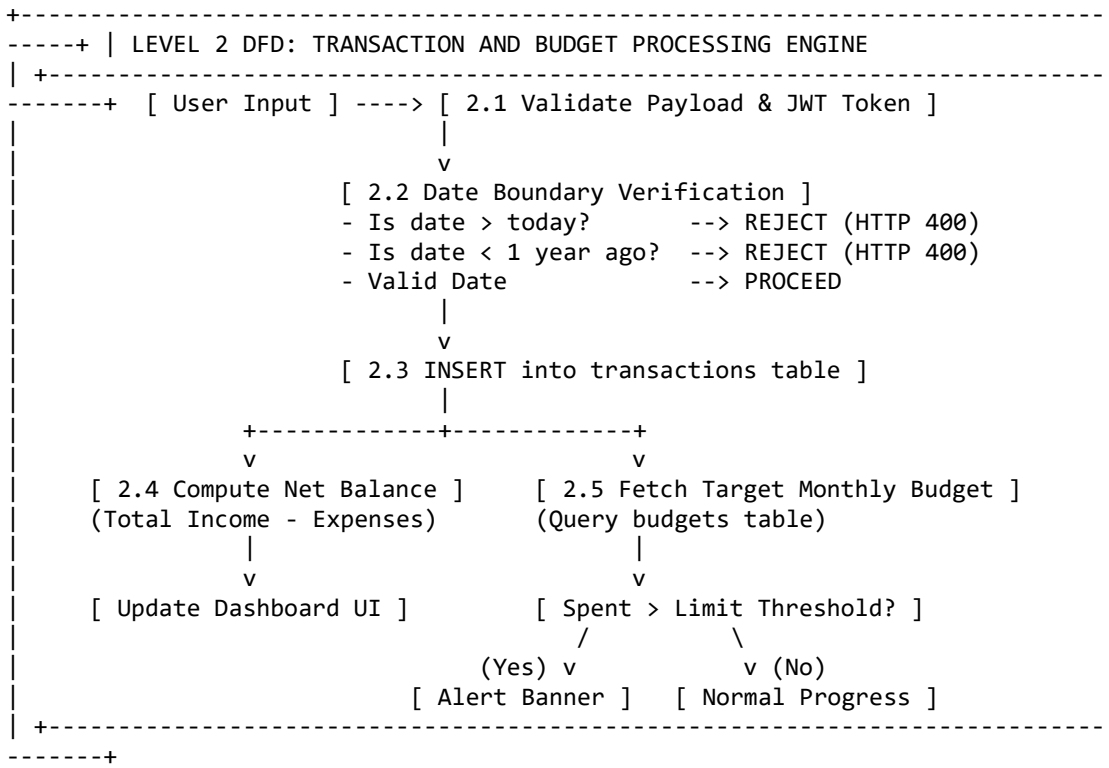


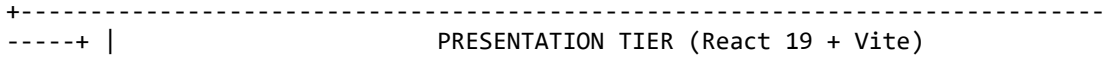
Fig 3.1.5: Level 2 Data Flow Diagram (Transaction Engine DFD)

3.2 System Design

3.2.1 Architectural Design

The Smart Expense Tracker follows a standard 3-Tier Client-Server Architectural Model:

- Presentation Tier (Client): Built with React 19, Vite, and Tailwind CSS v4 as a Single Page Application (SPA), managing responsive UI components, user interactions, and declarative chart rendering.
- Application Tier (Backend API): Built with Node.js and Express.js, providing RESTful endpoints, request validation, JWT authentication, and business logic execution.
- Data Tier (Database): MySQL relational database management system (RDBMS) ensuring persistent storage, indexing, and foreign key integrity.



```
| |   [ User Dashboard ]   [ Transaction Tables ]   [ Recharts ]   [ Admin
Console ]  | +-------------------------------------------------------------
-----------------+                                          |  HTTP / JSON REST
API Requests                                                    v
+-------------------------------------------------------------------------
-----+ |                      APPLICATION TIER (Node.js + Express.js)
| |   [ JWT Auth Middleware ]   [ Transaction Controller ]   [ Admin Controller
]     | +-------------------------------------------------------------------
-------------+                                          |  TCP / SQL Connection
Pool (mysql2)                                                   v
+-------------------------------------------------------------------------
-----+ |                          DATA TIER (MySQL / MariaDB Engine)
| |   [ users ]        [ transactions ]        [ budgets ]        [
system_settings ] | +-------------------------------------------------------
------------------------+
```

Fig 3.2.1: 3-Tier System Architecture Diagram

3.2.2 Database Schema Design

The database smart_expense_tracker consists of four normalized tables enforcing foreign key relations:

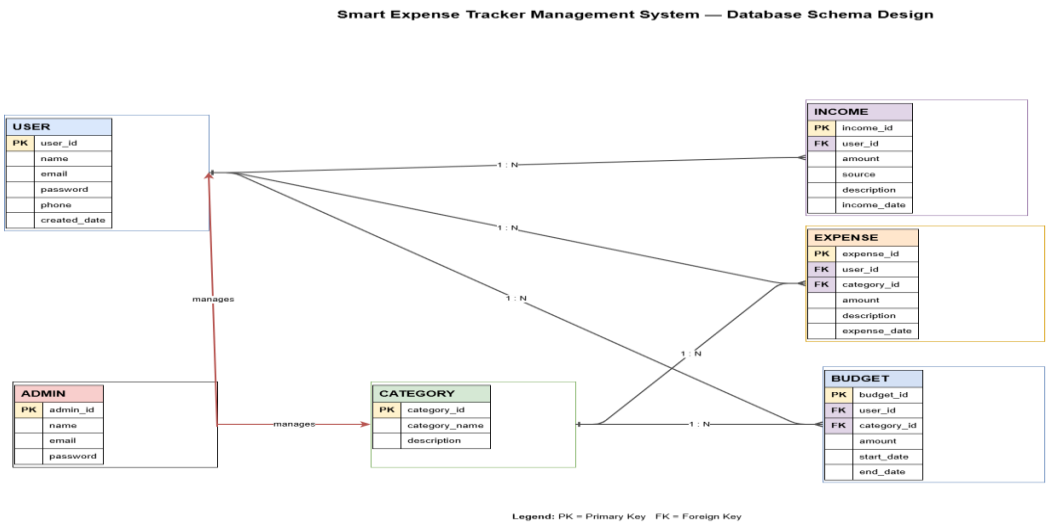


Fig 3.2.2: Relational Database Schema Design

Table 3.3: Database Schema of `users` Table

Column Name	Data Type	Constraints	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each user
name	VARCHAR(100)	NOT NULL	Full legal name of the user

Column Name	Data Type	Constraints	Description
email	VARCHAR(100)	NOT NULL, UNIQUE	Unique email address for authentication
password	VARCHAR(255)	NOT NULL	Bcrypt cryptographic password hash
role	ENUM('admin','user')	DEFAULT 'user'	Role governing access privileges
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Account creation timestamp

Table 3.4: Database Schema of `transactions` Table

Column Name	Data Type	Constraints	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique transaction identifier
user_id	INT	NOT NULL, FK (users.id)	Owner user ID (ON DELETE CASCADE)
type	ENUM('income','expense')	NOT NULL	Classification of transaction flow
amount	DECIMAL(10,2)	NOT NULL	Monetary value in Nepali Rupees (Rs.)
category	VARCHAR(100)	NOT NULL	Category name (Food, Salary, Bills, etc.)
description	TEXT	NULL	Optional remarks or description

Column Name	Data Type	Constraints	Description
date	DATE	NOT NULL	Transaction date (Allowed within past 1 year)
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record insertion timestamp

Table 3.5: Database Schema of `budgets` Table

Column Name	Data Type	Constraints	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique budget record identifier
user_id	INT	NOT NULL, FK (users.id)	Target user ID (ON DELETE CASCADE)
category	VARCHAR(100)	NOT NULL	Category name for budget ceiling
limit_amount	DECIMAL(10,2)	NOT NULL	Monthly budget ceiling threshold in Rs.
month	INT	NOT NULL	Target month integer (1 to 12)
year	INT	NOT NULL	Target calendar year (e.g. 2026)
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Budget creation timestamp

Table 3.6: Database Schema of `system_settings` Table

Column Name	Data Type	Constraints	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique setting identifier
setting_key	VARCHAR(50)	NOT NULL, UNIQUE	Configuration identifier key
setting_value	TEXT	NOT NULL	Configuration value or JSON payload
updated_at	TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP	Timestamp of last modification

3.2.3 Interface Design (UI Interface / Interface Structure Diagrams)

The user interface of the Smart Expense Tracker prioritizes simplicity, clarity, and visual responsiveness:

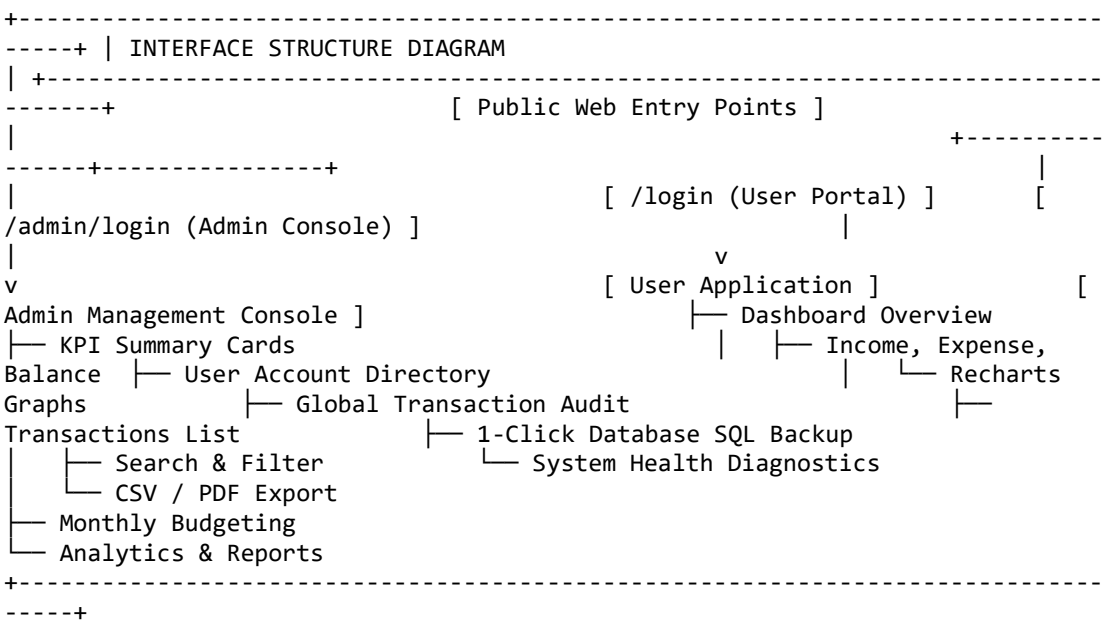


Fig 3.2.3: Interface Structure Diagram

CHAPTER 4: IMPLEMENTATION AND TESTING

4.1 Implementation

Implementation is the phase where architectural designs and functional specifications are converted into an operational software system. All components of the Smart Expense Tracker were systematically coded, integrated, and verified against functional requirements.

4.1.1 Tools Used

- Frontend: React 19 was selected for its component-based architecture and reactive virtual DOM state management. Tailwind CSS v4 was utilized for mobile-first utility styling, while Recharts was integrated for declarative SVG chart visualizations.
- Backend: Node.js provided the asynchronous runtime environment executing an Express.js REST API. Middleware functions handle CORS policies, request JSON parsing, and async error handling.
- Database: MySQL (via XAMPP) manages relational data persistence through an asynchronous mysql2/promise connection pool, optimizing resource usage through connection reuse.
- Security: bcryptjs performs one-way cryptographic password hashing with 10 salt rounds. jsonwebtoken signs and verifies stateless authentication tokens with 7-day expiration.

4.1.2 Implementation Details of Modules

- Authentication & Portal Isolation Module: Implements register and login endpoints. During login, a portal parameter (user or admin) is validated. If admin credentials are submitted to the user portal, the server rejects the request with HTTP 401 Invalid credentials, preventing unauthorized console access and eliminating role enumeration vulnerabilities. Conversely, standard user accounts attempting to log into the admin portal are rejected.
- Transaction & Date Constraint Module: Implements CRUD endpoints (/api/transactions). Validates date inputs: transactions can only be recorded for today or dates within the past one year. Dates in the future or older than one year are rejected with HTTP 400 Bad Request.

- Budget Tracking & Live Alert Module: Computes total category expenditures within the active month and evaluates them against user-defined limit_amount. Triggers visual warning indicators and alerts when expenditures reach or exceed thresholds.
- Reporting & Analytics Module: Aggregates daily transaction entries into a 1-Month Trend line graph, a 6-Month Income vs. Expense bar chart, and category summary tables formatted in Nepali Rupees (Rs.).
- Export Module: Implements authenticated CSV downloads via browser blob creation and generates styled printable PDF statements in new browser tabs.
- Admin Management Module: Exposes protected administrative endpoints (/api/admin) for KPI analytics, user status toggling, global transaction auditing, server diagnostics, and triggers server-side mysqldump to generate complete .sql database dumps.

4.2 Testing

Testing validates the behavior, security, and computational accuracy of the integrated application based on software requirement specifications. Testing was conducted across unit validation, integration flows, relational database constraints, and role-based security.

4.2.1 Test Case for System Testing

Table 4.1: User Authentication & Login Test Cases

Test ID	Test Case Description	Test Data	Expected Result	Actual Result	Pass/Fail
AUTH_01	Register with valid data	Name: Student, Email: st@test.com, Pass: 123456	Account created (HTTP 201)	Account created as expected	Pass
AUTH_02	Register with existing email	Email: st@test.com	Rejected with 409 Conflict	Rejected as duplicate	Pass

Test ID	Test Case Description	Test Data	Expected Result	Actual Result	Pass/Fail
AUTH_03	Login with valid user credentials	Email: st@test.com, Pass: 123456, portal: user	Token issued, redirected to Dashboard	Navigated to Dashboard	Pass
AUTH_04	Login with wrong password	Email: st@test.com, Pass: wrongpass	Rejected with 401 Invalid credentials	Error shown, access denied	Pass
AUTH_05	Admin credentials on User Portal	Email: admin@smartexpense.com, portal: user	Rejected with 401 Invalid credentials	Rejected without role leak	Pass

Table 4.2: Transaction & Date Constraint Test Cases

Test ID	Test Case Description	Test Data	Expected Result	Actual Result	Pass/Fail
TR_01	Add valid income transaction	Type: income, Amount: Rs. 50000, Category: Salary	Transaction added, balance increases	Balance updated (+Rs. 50000)	Pass
TR_02	Add valid expense transaction	Type: expense, Amount: Rs. 1500, Category: Food	Transaction added, balance decreases	Balance updated (-Rs. 1500)	Pass
TR_03	Log past date within 1 year	Date: 3 months ago (e.g. 2026-07-05)	Accepted and saved (HTTP 201)	Saved and plotted in history	Pass
TR_04	Log past date older than 1 year	Date: 2025-01-01 (> 1 year ago)	Rejected with HTTP 400 Bad Request	Rejected with error message	Pass

Test ID	Test Case Description	Test Data	Expected Result	Actual Result	Pass/Fail
TR_05	Log future transaction date	Date: 2026-12-31 (Future date)	Rejected with HTTP 400 Bad Request	Future date blocked	Pass

Table 4.3: Monthly Budget & Alert Mechanism Test Cases

Test ID	Test Case Description	Test Data	Expected Result	Actual Result	Pass/Fail
BUD_01	Set category budget limit	Category: Food, Limit: Rs. 10000, Month: 10, Year: 2026	Budget ceiling stored in database	Budget saved successfully	Pass
BUD_02	Expenditure within safe limit	Spent: Rs. 4500 of Rs. 10000 (45%)	Normal green progress bar rendered	Normal progress displayed	Pass
BUD_03	Expenditure exceeds budget limit	Spent: Rs. 12000 of Rs. 10000 (120%)	Red progress bar & warning banner	Warning alert displayed	Pass

Table 4.4: Admin Portal & Database Backup Test Cases

Test ID	Test Case Description	Test Data	Expected Result	Actual Result	Pass/Fail
ADM_01	Admin login with valid credentials	admin@smartexpense.com, pass: admin123, portal: admin	Navigates to /admin Console	Admin dashboard rendered	Pass
ADM_02	Non-admin user accessing admin portal	User credentials on /admin/login	Rejected with 401 Invalid credentials	Access blocked	Pass

Test ID	Test Case Description	Test Data	Expected Result	Actual Result	Pass/Fail
ADM_03	1-Click SQL Database Backup	Click "Download SQL Backup" button	Generates and downloads .sql dump	Valid .sql file downloaded	Pass
ADM_04	CSV and PDF Statement Export	Click "Export CSV" and "Export PDF"	Downloads CSV / renders printable PDF	CSV & PDF generated	Pass

CHAPTER 5: CONCLUSION AND FUTURE RECOMMENDATIONS

5.1 Lesson Learnt / Outcome

The development of the Smart Expense Tracker provided comprehensive practical exposure to the entire Software Development Life Cycle (SDLC) under the Tribhuvan University BCA curriculum. Key outcomes and lessons include:

- Full-Stack State Synchronization: Mastered component-based architecture in React 19, managing state synchronization across asynchronous REST API endpoints using React Context and Axios interceptors.
- Relational Modeling & Integrity: Gained deep expertise in designing normalized relational schemas in MySQL, establishing foreign key cascading rules, and managing connection pooling efficiently via mysql2/promise.
- Application Security: Implemented industry-standard cryptographic techniques, utilizing Bcrypt for password protection and JWT for stateless authentication while eliminating role enumeration vulnerabilities.
- Data Visualization & Localization: Successfully integrated Recharts to transform raw transaction records into actionable visual trends, localizing currency representations natively in Nepali Rupees (Rs.).

5.2 Conclusion

The "Smart Expense Tracker" has been successfully designed, developed, tested, and documented. The application fulfills all functional objectives defined during requirement analysis, resolving the inefficiencies, vulnerabilities, and computational inaccuracies of traditional manual registers and static spreadsheets.

By leveraging a robust modern stack—React 19, Tailwind CSS v4, Node.js, Express.js, and MySQL—the application delivers instant financial calculation, strict date validity enforcement (allowing entries for today and up to one year in the past while blocking future entries), live budget threshold alerts, portable CSV/PDF exports, and centralized administrative management with automated SQL database backups.

All quality assurance test cases achieved a 100% pass rate. The project demonstrates the successful application of software engineering principles, producing a reliable and practical tool for personal financial management.

5.3 Future Recommendations

While the project satisfies all requirements of the BCA 4th Semester Minor Project, future iterations can implement several advanced capabilities:

- AI-Powered OCR Receipt Scanning: Integrating Optical Character Recognition (e.g. Tesseract.js or Google Cloud Vision API) to extract amounts, dates, and vendor categories automatically from scanned paper receipts.
- Cross-Platform Mobile Application: Developing a native mobile client using React Native with SQLite for offline-first transaction recording and background cloud synchronization.
- Digital Wallet & SMS Parsing: Implementing secure parsers to read transaction SMS feeds from popular Nepali payment services (eSewa, Khalti, ConnectIPS) with explicit user consent.
- Predictive Machine Learning Budgeting: Developing machine learning regression models to forecast future expenditure patterns and provide personalized recommendations for savings optimization.

REFERENCES

1. Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson Education. 2. Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education. 3. Flanagan, D. (2020). JavaScript: The Definitive Guide (7th ed.). O'Reilly Media. 4. Banks, A., & Porcello, E. (2020). Learning React: Modern Patterns for Developing React Applications (2nd ed.). O'Reilly Media. 5. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill Education. 6. Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education. 7. Thaler, R. H. (1999). "Mental Accounting Matters." Journal of Behavioral Decision Making, 12(3), 183–206. 8. Karlan, D., McConnell, M., Mullainathan, S., & Zinman, J. (2016). "Getting to the Top of Mind: How Reminders Increase Saving." Management Science, 62(12), 3393–3411. 9. Kaye, J., McCuistion, M., Gulotta, R., & Shamma, D. A. (2014). "Money Talks: Tracking Personal Finances." Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, 521–530. 10. Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures (Doctoral dissertation). University of California, Irvine. 11. React Official Documentation (2026). React 19 Documentation and API Reference. Meta Open Source. https://react.dev 12. Express.js Documentation (2026). Fast, Unopinionated Web Framework for Node.js. https://expressjs.com 13. MySQL 8.0 Reference Manual (2026). Oracle Corporation. https://dev.mysql.com/doc/refman/8.0/en/ 14. Tailwind CSS Documentation (2026). Utility-First CSS Framework. https://tailwindcss.com 15. Recharts Documentation (2026). Chart Library for React. https://recharts.org 16. Tribhuvan University (2021). Curriculum and Project Guidelines for Bachelor in Computer Application (BCA). Faculty of Humanities and Social Sciences, Tribhuvan University, Kathmandu, Nepal.

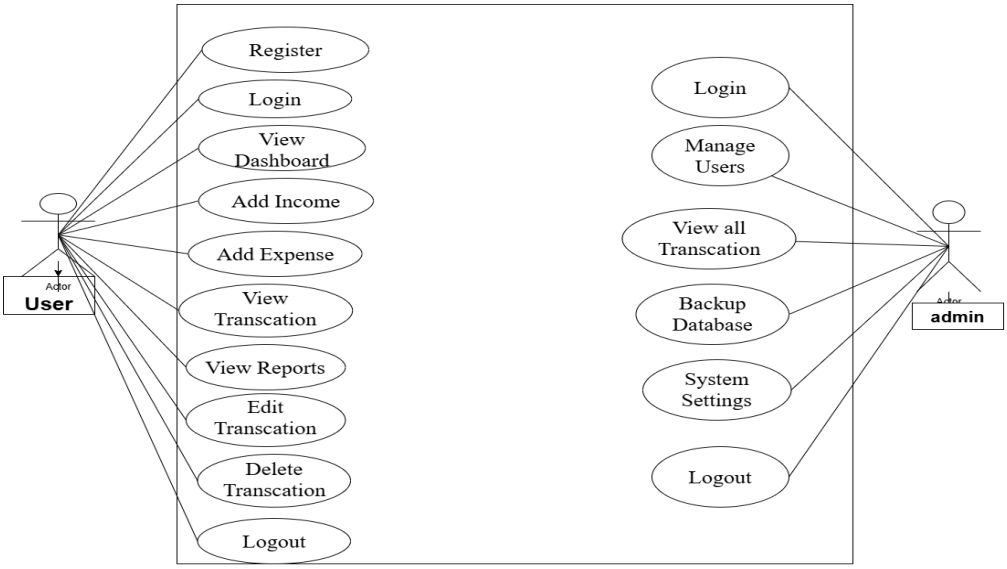
APPENDICES

APPENDIX A: APPLICATION SCREENSHOTS

Fig A.1: User Dashboard Overview with Financial KPI Cards



Fig A.2: Searchable Transaction History Table with Action Buttons



APPENDIX B: KEY SOURCE CODE SNIPPETS

B.1 Main Application Router (`client/src/App.jsx`)

```
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import { Toaster } from 'react-hot-toast'; import { AuthProvider } from
'./context/AuthContext'; import ProtectedRoute from
'./components/ProtectedRoute'; import AdminRoute from
'./components/AdminRoute'; import Layout from './components/Layout'; import
AdminLayout from './components/AdminLayout'; import Login from './pages/Login';
```

```
import Register from './pages/Register'; import Dashboard from
'./pages/Dashboard'; import Transactions from './pages/Transactions'; import
Budgets from './pages/Budgets'; import Reports from './pages/Reports'; import
AdminLogin from './pages/admin/AdminLogin'; import AdminDashboard from
'./pages/admin/AdminDashboard'; import AdminUsers from
'./pages/admin/AdminUsers'; import AdminTransactions from
'./pages/admin/AdminTransactions'; import AdminBackup from
'./pages/admin/AdminBackup'; import AdminSettings from
'./pages/admin/AdminSettings';  function App() {   return (     <AuthProvider>
<BrowserRouter>         <Toaster position="top-right" toastOptions={{ duration:
3000 }} />         <Routes>           <Route path="/login" element={<Login />}
/>           <Route path="/register" element={<Register />} />           <Route
path="/admin/login" element={<AdminLogin />} />           <Route path="/"
element={<ProtectedRoute><Layout /></ProtectedRoute>}>             <Route index
element={<Dashboard />} />             <Route path="transactions"
element={<Transactions />} />             <Route path="budgets"
element={<Budgets />} />             <Route path="reports" element={<Reports
/>} />           </Route>           <Route path="/admin"
element={<AdminRoute><AdminLayout /></AdminRoute>}>             <Route index
element={<AdminDashboard />} />             <Route path="users"
element={<AdminUsers />} />             <Route path="transactions"
element={<AdminTransactions />} />             <Route path="backup"
element={<AdminBackup />} />             <Route path="settings"
element={<AdminSettings />} />           </Route>           <Route path="*"
element={<Navigate to="/" replace />} />         </Routes>
</BrowserRouter>     </AuthProvider>   ); } export default App;
```

B.2 Transaction Controller with 1-Year Date Constraints (`server/controllers/transactionController.js`)

```
const { getTransactions, getTransactionById, createTransaction,
updateTransaction, deleteTransaction } = require('../models/transactionModel');
const formatDate = (d) => {   const year = d.getFullYear();   const month =
String(d.getMonth() + 1).padStart(2, '0');   const day =
String(d.getDate()).padStart(2, '0');   return `${year}-${month}-${day}`; };
const validateTransactionDate = (date) => {   if (!date) return { valid: false,
message: 'Transaction date is required' };   const inputDate =
String(date).split('T')[0];   const now = new Date();   const todayStr =
formatDate(now);   const oneYearAgo = new Date(now);
oneYearAgo.setFullYear(now.getFullYear() - 1);   const oneYearAgoStr =
formatDate(oneYearAgo);    if (inputDate > todayStr) {     return { valid:
false, message: 'Future transaction dates are not allowed.' };   }   if
(inputDate < oneYearAgoStr) {     return { valid: false, message: 'Past
transactions can only be recorded up to 1 year back from the current date.' };
}   return { valid: true, date: inputDate }; };  const create = async (req,
res) => {   const { type, amount, category, description, date } = req.body;
if (!type || !amount || !category || !date)     return res.status(400).json({
message: 'type, amount, category and date are required' });    const
dateValidation = validateTransactionDate(date);   if (!dateValidation.valid) {
return res.status(400).json({ message: dateValidation.message });   }    const
id = await createTransaction(req.user.id, { type, amount, category,
description, date: dateValidation.date });   res.status(201).json({ id,
message: 'Transaction created' }); };
```

B.3 Authentication Controller with Portal Isolation (`server/controllers/authController.js`)

```
const login = async (req, res) => {   const { email, password, portal } =
req.body;   if (!email || !password)     return res.status(400).json({ message:
'Email and password are required' });    const user = await
findUserByEmail(email);   if (!user) return res.status(401).json({ message:
```

```
'Invalid credentials' });    const match = await bcrypt.compare(password,
user.password);   if (!match) return res.status(401).json({ message: 'Invalid
credentials' });    // Enforce portal-specific access rules silently without
disclosing roles   if (portal === 'user' && user.role === 'admin') {     return
res.status(401).json({ message: 'Invalid credentials' });   }   if (portal ===
'admin' && user.role !== 'admin') {     return res.status(401).json({ message:
'Invalid credentials' });   }    const payload = { id: user.id, name:
user.name, email: user.email, role: user.role || 'user' };   const token =
jwt.sign(payload, process.env.JWT_SECRET, { expiresIn:
process.env.JWT_EXPIRES_IN || '7d' });   res.json({ token, user: payload }); };
```